

Managing Sessions in Claude Code

Companion to your live demo — take this back to your desk and use it

What you saw in the demo

In about 20 minutes you watched a walkthrough of how Claude Code remembers your work across crashes, lunch breaks, and week-long pauses. The four ideas worth carrying away:

1. **Sessions persist automatically.** Every conversation is saved to disk by default — you don't have to opt in.
2. **-c brings the most recent one back.** Two keystrokes to resume yesterday's debugging session with full context.
3. **Context isn't free.** `/cost` , `/compact` , and `/clear` let you control token spend and keep responses sharp.
4. **Sessions persist the conversation; checkpoints persist the files.** Two separate systems, two separate rollback commands.

The rest of this guide walks through each step that was demonstrated, in the same order, so you can recreate it at your own pace.

Starting and ending a session

Starting a fresh session

```
cd ~/projects/your-app
claude
```

This opens an interactive Claude Code session in the directory you're standing in. Two things happen automatically:

- The project directory becomes the working context
- If a `CLAUDE.md` exists in that directory, it loads into context immediately

The second part is why we use `CLAUDE.md` . It pre-loads the durable parts of your prompt — project conventions, file layout, build commands, testing patterns — so every session starts knowing your project. You don't have to mention any of it.

Verifying CLAUDE.md auto-loaded

```
> what's in our CLAUDE.md?
```

Claude summarizes the contents without you having mentioned the file by name. That's proof the loading happened automatically.

Exiting

Press **Ctrl+D** or type `/exit` .

You don't have to "save" anything. Claude Code wrote the session to disk while you were working. It's already there when you exit.

Resuming what you were doing

Continue the most recent session

```
claude --continue
# or the short form:
claude -c
```

This is your daily go-to. It picks up your most recent session in the current directory — full conversation, every tool result, every file Claude touched. Use it after lunch, after a crash, after closing the terminal by mistake.

Pick from a list

```
claude --resume
# or:
claude -r
```

Opens an interactive picker with your past sessions, showing message counts, timestamps, and the git branch each was on.

Shortcuts inside the picker:

Key	What it does
B	Filter to sessions on the current git branch
A	Toggle between "this directory" and "all projects"
/	Search past conversations by keyword
↑ / ↓	Move through the list
Enter	Open the selected session

Name a session so you can find it again

```
> /rename auth-jwt-migration
```

The session now shows up in the picker as `auth-jwt-migration` instead of an auto-generated summary. Three days from now you will not remember what "discussing the auth changes and refactoring" was. You will remember "JWT migration."

Switch sessions without exiting

```
> /resume
```

The picker opens inside your running session. Useful when you're bouncing between tasks in a single working block.

Managing context and cost

This section matters most for keeping Claude Code productive — and affordable — over long sessions.

Check current cost

```
> /cost
```

Shows token usage broken down by input, output, and cache hits, plus the dollar cost of this session so far. Run it when responses start feeling sluggish, or when you've been in one session for a while.

Visualize how full the context window is

```
> /context
```

Displays a colored grid showing what's filling the context window. When the grid is mostly full, Claude is starting to lose earlier context — that's your signal to compact or clear.

Compact a long conversation

```
> /compact
# or with a focus:
> /compact focus on the auth changes
```

Summarizes the conversation, replacing the back-and-forth with a condensed version that keeps the gist. Token count drops sharply. You can add a focus phrase to bias what's preserved.

Use it after an exploration phase, when only the decisions matter and the dead ends don't.

Hard reset — clear the conversation

```
> /clear
```

Wipes the conversation history completely. **CLAUDE.md stays loaded** — only the back-and-forth is gone. Use this when you've finished one task and want to start fresh in the same session without losing your project context.

Roll back file changes

```
> /rewind
# or press Escape twice:
[Esc][Esc]
```

Opens the checkpoint menu — every file Claude changed during this session is rollback-able.

This is the most important distinction from the demo: sessions persist the **conversation**; checkpoints persist the **filesystem**. They're separate.

- `--continue` brings back the conversation — it does *not* undo file changes

- `/rewind` walks back files — the conversation still remembers what was tried

Use both together. If Claude made a change you don't want, `/rewind` the file, then ask Claude to try a different approach. It still has context from the first attempt.

Power moves

Run one prompt and exit (headless mode)

```
claude -p "summarize what changed in the last 5 commits"
```

Single response, no interactive shell, exits when done. Doesn't save a session. This is the mode for:

- CI pipelines and GitHub Actions
- Pre-commit hooks
- Cron jobs and scheduled tasks
- Shell scripts where you want a one-shot answer

If you want history, use interactive mode instead.

Resume and immediately prompt

```
claude -c "now also check the test files"
```

Picks up your last session and runs the new prompt in one command. Useful in scripts, or for fast follow-ups from the command line.

Where sessions live on disk

```
ls ~/.claude/projects/  
ls ~/.claude/sessions/
```

Sessions are JSON files in your home directory, organized per-project. You can back them up. You can audit them for compliance. You can copy them between machines.

Gotchas to remember

Gotcha	Why it bites
<code>--continue</code> won't undo file changes	Sessions persist the conversation, not the filesystem. Use <code>/rewind</code> for files.
<code>/clear</code> doesn't reload an edited <code>CLAUDE.md</code>	If you change <code>CLAUDE.md</code> mid-session, restart Claude Code to pick it up.
Headless mode (<code>-p</code>) doesn't save	Don't use it for work you might want to resume — it's fire-and-forget by design.
Compacting is lossy	The summary keeps the gist, not every detail. Don't compact if you need exact earlier wording.

Long-running sessions accumulate drift	After 4–5 days of compacting, start a fresh session. Cleaner than fighting accumulated noise.
--	---

Cheat sheet

Starting

Command	What it does
<code>claude</code>	Start a session in the current directory
<code>claude "prompt"</code>	Start a session with an initial prompt
<code>claude -p "prompt"</code>	Headless / one-shot — no interactive mode

Resuming

Command	What it does
<code>claude -c / --continue</code>	Resume the most recent session in this directory
<code>claude -r / --resume</code>	Open the interactive picker
<code>claude -r <session-id></code>	Jump straight to a specific session by ID
<code>claude -c "prompt"</code>	Resume and send a new prompt in one shot

Inside a session

Slash command	What it does
<code>/resume</code>	Switch sessions without exiting
<code>/rename <name></code>	Give this session a human-readable label
<code>/clear</code>	Wipe conversation history (CLAUDE.md stays loaded)
<code>/compact</code>	Summarize history to save tokens
<code>/compact focus on X</code>	Compact, biased toward topic X
<code>/cost</code>	Show token usage and dollar cost
<code>/context</code>	Visualize how full the context window is
<code>/rewind</code> (OR <code>Esc Esc</code>)	Roll back file changes via checkpoints
<code>/exit</code> (OR <code>Ctrl+D</code>)	Exit — session auto-saves

Where things live on disk

Path	Contents
<code>~/.claude/projects/</code>	Per-project session files (JSON)

~/ .claude/sessions/	Session UUIDs and metadata
~/ .claude/history.jsonl	Aggregate conversation history
<project>/CLAUDE.md	Project-specific context loaded every session

Three things to try first

When you get back to your desk:

1. **Run `claude -c` in a project you used yesterday.** Watch your previous work come back. This is the muscle memory you want to build.
2. **Name your next important session with `/rename`.** Then re-open the picker and see how much easier it is to find.
3. **Run `/cost` mid-session at least once today.** Just to see what your work actually costs. Most teams are surprised.

Further reading

- [Claude Code documentation — overview](#)
- [Working with sessions](#)
- [Steve Kinney — Claude Code session management](#)

Prepared for Training programs · Technizer India